

Karatsuba Multiplication

Fast Integer Multiplication via Divide and Conquer

Week 1 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

- 1 Motivation
- 2 The Naive Algorithm and Its Cost
- 3 Divide and Conquer: The Idea
- 4 Karatsuba's Trick
- 5 Recurrence and Running Time
- 6 Worked Running Example
- 7 Algorithm and Python Implementation
- 8 Comparing the Two Algorithms
- 9 Summary

A question that sounds too simple to matter

Question

How fast can we multiply two n -digit integers?

- This is the algorithm you learned in *grade school*: line up the digits, multiply, shift, add.
- It feels “obviously” as fast as multiplication can be.
- Yet in 1960, a 23-year-old student named **Anatoly Karatsuba**, in a seminar run by Andrey Kolmogorov, found a faster algorithm — and disproved a conjecture that $\Theta(n^2)$ was optimal.

Why we care (course-wide theme)

This is our **first non-trivial divide-and-conquer algorithm**: it shows that “obviously optimal” brute force is often *not* optimal, and that a clever recombination step can change the exponent in the running time, not just the constant.

Where do large multiplications actually matter?

- **Cryptography:** RSA works with numbers that have hundreds or thousands of digits (bits).
- **Computer algebra systems:** exact arbitrary-precision arithmetic (Python's `int`, Java's `BigInteger`).
- **Numerical / scientific computing:** high-precision constants (π to trillions of digits), polynomial and matrix multiplication (same divide-and-conquer idea reused).

For *small* numbers (say, 64-bit machine integers), the constant-factor overhead of Karatsuba is not worth it — hardware multiplies those in $O(1)$. Karatsuba only pays off once n (the number of digits) is large enough that the *asymptotic* exponent dominates the constants. This is a recurring lesson: *asymptotic analysis tells you what wins eventually, not what wins for every input size.*

Grade-school multiplication, counted carefully

To multiply two n -digit numbers x and y by hand:

- Multiply x by *each* of the n digits of y separately $\Rightarrow n$ partial products.
- Each partial product costs $O(n)$ single-digit multiplications (with carries).
- Add up the n shifted partial products: $O(n)$ more work per addition, $O(n)$ additions.

$$\underbrace{n}_{\text{partial products}} \times \underbrace{O(n)}_{\text{cost each}} = \Theta(n^2) \text{ single-digit multiplications.}$$

Key fact

The grade-school algorithm is $\boxed{\Theta(n^2)}$. Doubling the number of digits quadruples the work.

Question this section answers: can we multiply two n -digit numbers using asymptotically *fewer* than $\Theta(n^2)$ single-digit multiplications?

Step 1: split each number in half

Let x and y be n -digit numbers (assume n is even; pad with a leading zero if not). Split each into a high half and a low half, each with $n/2$ digits:

$$x = x_1 \cdot 10^{n/2} + x_0, \quad y = y_1 \cdot 10^{n/2} + y_0.$$



high $n/2$ digits low $n/2$ digits

Example: $x = 1234$, $n = 4$, $n/2 = 2 \Rightarrow x_1 = 12$, $x_0 = 34$, and indeed $12 \cdot 10^2 + 34 = 1234$. ✓

Step 2: expand the product

Multiply out $xy = (x_1 \cdot 10^{n/2} + x_0)(y_1 \cdot 10^{n/2} + y_0)$:

$$xy = \underbrace{x_1 y_1}_{z_2} \cdot 10^n + \underbrace{(x_1 y_0 + x_0 y_1)}_{z_1} \cdot 10^{n/2} + \underbrace{x_0 y_0}_{z_0}.$$

This expresses the product of two n -digit numbers in terms of products of $n/2$ -**digit** numbers: $x_1 y_1$, $x_1 y_0$, $x_0 y_1$, $x_0 y_0$.

The naive divide-and-conquer attempt

Compute all **four** products $x_1 y_1$, $x_1 y_0$, $x_0 y_1$, $x_0 y_0$ recursively, then combine with shifts and additions (which cost only $O(n)$).

Why four recursive calls is *not* good enough

Let $T(n)$ be the number of single-digit multiplications to multiply two n -digit numbers this way. Four recursive calls on half-size inputs, plus $O(n)$ combination work:

$$T(n) = 4 T(n/2) + O(n).$$

By the Master theorem ($a = 4$, $b = 2$, $\log_b a = \log_2 4 = 2$):

$$T(n) = \Theta(n^2).$$

No improvement!

Splitting the numbers in half and recursing on **four** sub-products gives back exactly the same $\Theta(n^2)$ bound as grade-school multiplication. Divide and conquer, by itself, bought us nothing here — *something smarter is needed*.

The key idea: trade a multiplication for additions

We do not need x_1y_0 and x_0y_1 *individually* — only their **sum** $z_1 = x_1y_0 + x_0y_1$. Karatsuba's observation:

$$(x_1 + x_0)(y_1 + y_0) = x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0.$$

Rearranging gives the cross term $x_1y_0 + x_0y_1$ directly:

Karatsuba's identity

$$z_1 = x_1y_0 + x_0y_1 = \underbrace{(x_1 + x_0)(y_1 + y_0)}_{\text{one new product}} - \underbrace{x_1y_1}_{z_2} - \underbrace{x_0y_0}_{z_0}.$$

We already need to compute $z_2 = x_1y_1$ and $z_0 = x_0y_0$ for the other terms of the product. So z_1 costs **one extra multiplication** — $(x_1 + x_0)(y_1 + y_0)$ — plus a few $O(n)$ additions/subtractions, *instead of* the two multiplications x_1y_0 and x_0y_1 .

From four multiplications down to three

	Naive divide & conquer	Karatsuba
Recursive products	$x_1y_1, x_1y_0, x_0y_1, x_0y_0$	$x_1y_1, x_0y_0, (x_1+x_0)(y_1+y_0)$
Count	4	3

Given the three products $z_2 = x_1y_1$, $z_0 = x_0y_0$, and $p = (x_1 + x_0)(y_1 + y_0)$, recover z_1 and assemble the answer:

$$z_1 = p - z_2 - z_0, \quad xy = z_2 \cdot 10^n + z_1 \cdot 10^{n/2} + z_0.$$

All the additions, subtractions, and shifts (multiplying by a power of 10) cost only $O(n)$ total.
Only three multiplications of half-size numbers are ever needed.

Solving the recurrence

Karatsuba's algorithm makes **3** recursive calls on inputs of size $n/2$, plus $O(n)$ work to form the sums $(x_1 + x_0)$, $(y_1 + y_0)$ and to combine results:

$$T(n) = 3 T(n/2) + O(n), \quad T(1) = O(1).$$

By the **Master theorem** with $a = 3$, $b = 2$, $f(n) = O(n)$:

$$\log_b a = \log_2 3 \approx 1.585 > 1 = \deg f(n)$$

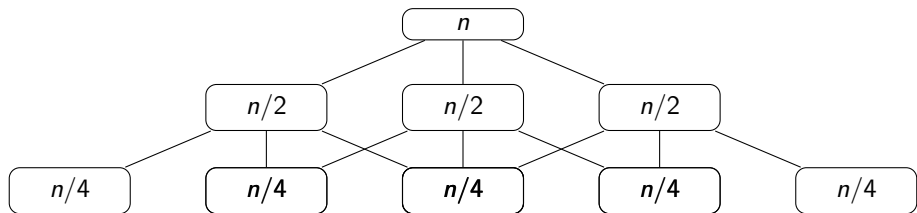
so we are in the “recursion dominates” case, giving

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta(n^{1.585}).$$

Why this matters

$n^{1.585}$ grows *strictly slower* than n^2 . For $n = 1024$ digits, $n^2 \approx 1.05 \times 10^6$ while $n^{1.585} \approx 3.2 \times 10^4$ — roughly **30× fewer** single-digit multiplications.

Recursion tree intuition



- Branching factor 3 (not 4): each level has 3^k nodes of size $n/2^k$.
- Work per node is $O(\text{node size})$, so level k costs $3^k \cdot O(n/2^k) = O(n) \cdot (3/2)^k$.
- Levels *grow geometrically* ($3/2 > 1$), so the **last level** (about $\log_2 n$ levels down, $n^{\log_2 3}$ leaves) dominates the total — exactly what the Master theorem tells us algebraically.

Running example: 1234×5678

Let $x = 1234$, $y = 5678$, so $n = 4$ and $n/2 = 2$.

$$x_1 = 12,$$

$$x_0 = 34,$$

$$y_1 = 56,$$

$$y_0 = 78.$$

Step 1 – two “easy” recursive products:

$$z_2 = x_1 y_1 = 12 \times 56 = 672, \quad z_0 = x_0 y_0 = 34 \times 78 = 2652.$$

Step 2 – the one “clever” recursive product:

$$x_1 + x_0 = 46, \quad y_1 + y_0 = 134, \quad p = 46 \times 134 = 6164.$$

Running example (continued): combine

Step 3 – recover the cross term:

$$z_1 = p - z_2 - z_0 = 6164 - 672 - 2652 = 2840.$$

Step 4 – assemble the final answer:

$$xy = z_2 \cdot 10^4 + z_1 \cdot 10^2 + z_0 = 672 \times 10000 + 2840 \times 100 + 2652.$$

$$xy = 6,720,000 + 284,000 + 2,652 = \boxed{7,006,652}.$$

Sanity check

Direct multiplication: $1234 \times 5678 = 7,006,652$. ✓ Matches exactly, using only **three** multiplications of 2-digit numbers instead of four.

Each of z_2 , z_0 , and p above would themselves be computed recursively by the same method once the numbers are large; here they are small enough to read off directly (the recursion's base case).

Pseudocode

```
1: function KARATSUBA( $x, y$ )
2:   if  $x < 10$  or  $y < 10$  then
3:     return  $x \cdot y$ 
4:   end if
5:    $n \leftarrow \max(\text{NUMDIGITS}(x), \text{NUMDIGITS}(y))$ 
6:    $m \leftarrow \lceil n/2 \rceil$ 
7:    $x_1, x_0 \leftarrow \lfloor x/10^m \rfloor, x \bmod 10^m$ 
8:    $y_1, y_0 \leftarrow \lfloor y/10^m \rfloor, y \bmod 10^m$ 
9:    $z_2 \leftarrow \text{KARATSUBA}(x_1, y_1)$ 
10:   $z_0 \leftarrow \text{KARATSUBA}(x_0, y_0)$ 
11:   $p \leftarrow \text{KARATSUBA}(x_1 + x_0, y_1 + y_0)$ 
12:   $z_1 \leftarrow p - z_2 - z_0$ 
13:  return  $z_2 \cdot 10^{2m} + z_1 \cdot 10^m + z_0$ 
14: end function
```

▷ base case

Python implementation

```
1 def karatsuba(x: int, y: int) -> int:
2     # Base case: single-digit multiplication.
3     if x < 10 or y < 10:
4         return x * y
5
6     n = max(len(str(x)), len(str(y)))
7     m = n // 2                                # split point (use ceil for odd n)
8     if n % 2:
9         m += 1
10
11     high_x, low_x = divmod(x, 10 ** m)
12     high_y, low_y = divmod(y, 10 ** m)
13
14     z2 = karatsuba(high_x, high_y)             # x1 * y1
15     z0 = karatsuba(low_x, low_y)              # x0 * y0
16     p = karatsuba(high_x + low_x,
17                   high_y + low_y)             # (x1+x0)(y1+y0)
18     z1 = p - z2 - z0                          # x1*y0 + x0*y1
19
20     return z2 * 10 ** (2 * m) + z1 * 10 ** m + z0
```

Testing it against the running example

```
1 >>> karatsuba(1234, 5678)
2 7006652
3 >>> 1234 * 5678
4 7006652
5 >>> karatsuba(1234, 5678) == 1234 * 5678
6 True
7
8 >>> import random
9 >>> all(karatsuba(a, b) == a * b
10 ...     for a, b in ((random.randint(0, 10**50),
11 ...                     random.randint(0, 10**50))
12 ...                     for _ in range(1000)))
13 True
```

This is exactly the spirit of **Problem 15** in this week's practical exercises: implement Karatsuba, then instrument it (count single-digit multiplications) and compare against the naive $\Theta(n^2)$ algorithm as n grows.

Naive vs. Karatsuba: growth of single-digit multiplications

Digits n	Naive $\Theta(n^2)$	Karatsuba $\Theta(n^{1.585})$	Speedup
10	100	≈ 39	$2.6\times$
100	10,000	$\approx 1,555$	$6.4\times$
1,000	1,000,000	$\approx 61,824$	$16\times$
10,000	100,000,000	$\approx 2,459,942$	$41\times$

- The gap *widens* as n grows – exactly what “different exponent” means.
- In practice, real libraries (e.g., Python’s `int`, GMP) switch from schoolbook multiplication to Karatsuba once operands exceed a threshold (tens of digits), and to even faster methods (Toom–Cook, Schönhage–Strassen, using FFTs) for very large operands – the same divide-and-conquer philosophy, pushed further.

Summary

- ① Grade-school multiplication of n -digit numbers is $\Theta(n^2)$.
- ② Splitting numbers in half and recursing on all **4** cross-products (naive divide and conquer) is *still* $\Theta(n^2)$ – splitting alone is not enough.
- ③ Karatsuba's identity $x_1y_0 + x_0y_1 = (x_1+x_0)(y_1+y_0) - x_1y_1 - x_0y_0$ trades two multiplications for one multiplication plus $O(n)$ extra additions, cutting the branching factor from 4 to **3**.
- ④ The recurrence $T(n) = 3T(n/2) + O(n)$ solves to $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$, asymptotically better than $\Theta(n^2)$.
- ⑤ This is our first concrete example that **divide-and-conquer is not automatically better** – the *combine* step must be designed cleverly (fewer recursive calls, not just smaller ones).

Looking ahead

We revisit divide-and-conquer formally, with the Master theorem proved in full, in Weeks 6–7. Karatsuba is the template: split, recurse, and find a combine step that minimizes the number of recursive calls.